

Global Query Filters in Entity Framework Core

In this article, I will discuss **Global Query Filters in Entity Framework Core (EF Core)** with Examples. Please read our previous article discussing [Shadow Properties in Entity Framework Core](#) with Examples.

What are Global Query Filters in Entity Framework Core?

Global Query Filters in Entity Framework Core allow developers to define filters at the model level that are automatically applied to all queries involving a specific entity type. These filters are defined once in the `OnModelCreating` method of our `DbContext` class, ensuring consistency and reducing repetitive code across our application. Once defined, these filters are automatically applied to all queries targeting those entities unless explicitly overridden.

They help implement features such as soft deletes, multi-tenancy, data partitioning by user roles, and other scenarios where certain conditions should consistently apply across all queries involving specific entities.

Key Features Global Query Filters in EF Core:

- **Automatic Filtering:** You don't need to apply the filter manually every time you query the database; it is automatically included in every query unless explicitly bypassed using LINQ's `IgnoreQueryFilters()` method.
- **Reusable:** Once defined, the filter applies globally to the entity.
- **Global Scope:** Filters apply to all queries, including Select, Include, Count, Sum, etc.
- **Custom Logic:** Global Query Filter allows us to define custom filtering logic, including conditions based on specific columns or global application settings.
- **Optional or Conditional:** Filters can be added or removed at runtime, depending on the needs of your application.

When to Use Global Query Filters

This is particularly useful for scenarios such as:

- **Soft Deletes:** When an entity is marked as deleted but should still exist in the database for historical purposes, a global query filter can exclude those entities from queries (e.g., skip all rows where `IsDeleted = true`).
- **Multi-Tenancy:** For applications serving multiple tenants, a filter can ensure that each tenant only sees their own data (e.g., filter by `TenantId` to retrieve data belonging to the current tenant).
- **Permissions/Access Control:** Filter out data that the current user should not have access to based on their roles or permissions. That means restricting data access based on user roles or permissions.

Soft Delete Functionality in EF Core Using Global Query Filters

In many Real-time applications, we implement soft deletes, where records are marked as deleted rather than physically removed from the database. This allows for data recovery and auditing while keeping the data out of regular application workflows.

Define the Entity (Order) with Soft Delete Property

We will implement a “**SOFT DELETE**” mechanism where entities marked as “deleted” are not physically removed from the database but instead excluded from query results.

In this example, assume you have an `Order` entity with an `IsDeleted` flag. Instead of manually filtering out deleted orders every time, we can set up a global query filter that automatically excludes orders marked as deleted. So, create a class file named **Order.cs** within the **Entities** folder and copy and paste the following code.

```

namespace EFCoreCodeFirstDemo.Entities
{
    public class Order
    {
        public int OrderId { get; set; }           // Primary Key
        public string ProductName { get; set; }    // Name of the product
        public int Quantity { get; set; }          // Quantity ordered
        public bool IsDeleted { get; set; }         // Soft delete flag
        public DateTime OrderDate { get; set; }    // Date of the order
    }
}

```

Configure Global Query Filter in DbContext

To implement global query filters, we need to override the `OnModelCreating` method of our `DbContext` class and use the `HasQueryFilter` method on the entity configurations. We will define a global query filter to exclude orders marked as deleted. So, modify the `EFCoreDbContext` class as follows:

```

using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        // Configuring the database connection and logging
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Display the generated SQL queries in the console
            optionsBuilder.LogTo(Console.WriteLine, LogLevel.Information);
            // Configuring the database connection
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        // Defining the global query filter in OnModelCreating
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // Global Query Filter to exclude soft-deleted orders
            modelBuilder.Entity<Order>().HasQueryFilter(o => !o.IsDeleted);
        }
        // DbSet representing Orders table
        public DbSet<Order> Orders { get; set; }
    }
}

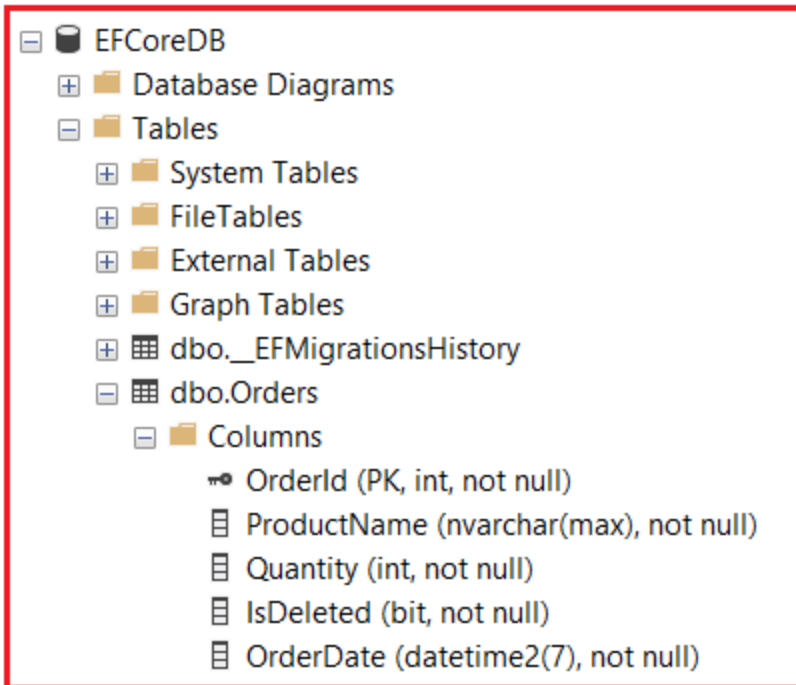
```

Apply Migrations and Update the Database

Open the Package Manager Console and execute the following command:

- **Add-Migration AddSoftDeleteToOrder**
- **Update-Database**

Once you execute the above commands, it should create the database with the required Orders table as shown in the below image:



Insert Sample Data

Please execute the following SQL commands to insert sample data into the Orders table. Ensure that some records are marked as deleted.

```
INSERT INTO Orders (ProductName, Quantity, IsDeleted, OrderDate) VALUES ('Laptop', 2, 0, GETDATE()); -- Not Deleted
INSERT INTO Orders (ProductName, Quantity, IsDeleted, OrderDate) VALUES ('Smartphone', 5, 1, GETDATE()); -- Deleted
INSERT INTO Orders (ProductName, Quantity, IsDeleted, OrderDate) VALUES ('Headphones', 10, 0, GETDATE()); -- Not Deleted
INSERT INTO Orders (ProductName, Quantity, IsDeleted, OrderDate) VALUES ('Monitor', 3, 1, GETDATE()); -- Deleted
```

Querying Entities with Global Query Filter Applied

Now, whenever you query the Orders DbSet, only orders not marked as deleted will be returned, and we don't need to add the condition in our queries explicitly. For a better understanding, please modify the Program class as follows:

```
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        public static void Main(string[] args)
        {
            try
            {
```

```

        using var context = new EFCoreDbContext();
        // Retrieve all active (non-deleted) orders
        var activeOrders = context.Orders.ToList();
        Console.WriteLine("\nActive Orders:");
        foreach (var order in activeOrders)
        {
            Console.WriteLine($"    \tOrder ID: {order.OrderId}, Product:
{order.ProductName}, Quantity: {order.Quantity}, Order Date:
{order.OrderDate}");
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"An error occurred while fetching active
orders: {ex.Message}");
    }
}
}
}

```

Output:

```

info: 26-09-2024 15:20:39.230 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (11ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT [o].[OrderId], [o].[IsDeleted], [o].[OrderDate], [o].[ProductName], [o].[Quantity]
      FROM [Orders] AS [o]
      WHERE [o].[IsDeleted] = CAST(0 AS bit)

Active Orders:
    Order ID: 1, Product: Laptop, Quantity: 2, Order Date: 26-09-2024 15:18:19
    Order ID: 3, Product: Headphones, Quantity: 10, Order Date: 26-09-2024 15:18:19

```

You can see it only displays orders where IsDeleted is false, and the same is applied in the WHERE clause of the SELECT query by EF Core.

Bypassing the Global Query Filter in Entity Framework Core

There are scenarios where you might need to ignore global query filters (e.g., an admin page that displays all records, regardless of IsDeleted). To disable the filters, we can use the `IgnoreQueryFilters()` extension method on our queries.

For example, we want to access all orders, including those marked as deleted (e.g., for administrative purposes). In that case, we can bypass the global query filter using the **`IgnoreQueryFilters`** method. For a better understanding, please modify the Program class as follows:

```

using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        public static void Main(string[] args)
        {
            try
            {

```

```

using var context = new EFCoreDbContext();
// Retrieve all orders, including deleted ones
var allOrders = context.Orders.IgnoreQueryFilters().ToList();
Console.WriteLine("\nAll Orders (Including Deleted):");
foreach (var order in allOrders)
{
    Console.WriteLine($"    \tOrder ID: {order.OrderId}, Product:
{order.ProductName}, Quantity: {order.Quantity}, Order Date:
{order.OrderDate}");
}
}
catch (Exception ex)
{
    Console.WriteLine($"An error occurred while fetching active
orders: {ex.Message}");
}
}
}
}

```

Output:

```

info: 26-09-2024 15:23:17.204 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (26ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT [o].[OrderId], [o].[IsDeleted], [o].[OrderDate], [o].[ProductName], [o].[Quantity]
      FROM [Orders] AS [o]

All Orders (Including Deleted):
    Order ID: 1, Product: Laptop, Quantity: 2, Order Date: 26-09-2024 15:18:19
    Order ID: 2, Product: Smartphone, Quantity: 5, Order Date: 26-09-2024 15:18:19
    Order ID: 3, Product: Headphones, Quantity: 10, Order Date: 26-09-2024 15:18:19
    Order ID: 4, Product: Monitor, Quantity: 3, Order Date: 26-09-2024 15:18:19

```

As you can see in the above output, all orders are displayed regardless of the IsDeleted flag.

Performance Consideration when working with Global Query Filters in EF Core

For optimal performance, especially with large datasets, consider adding an index to the IsDeleted column. This can significantly speed up query execution by allowing the database to locate records based on the soft delete flag quickly.

CREATE INDEX IX_Orders_IsDeleted ON Orders(IsDeleted);

Parameterized Global Filter in Entity Framework Core

EF Core supports parameterized filters, allowing us to pass runtime parameters into global query filters. This is useful for scenarios like multi-tenancy, where the tenant ID (Customer ID) is determined at runtime. Multi-tenancy refers to a software architecture where a single instance of an application (including its infrastructure, such as databases and servers) serves multiple customers, known as tenants.

Let's consider an example of a multi-tenant system where each customer can only view their own orders. This is done by filtering data based on the current CustomerId. We will use Customer and Order entities, and the Order entity will include a CustomerId property to identify the customer to which the order belongs. So, let us first create the entities:

Customer Entity:

So, create a class file named **Customer.cs** within the Entities folder and then copy and paste the following code.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Customer
    {
        public int CustomerId { get; set; } // Primary Key
        public string Name { get; set; } // Customer name
        public string Email { get; set; } // Customer email
        // One-to-many relationship: A customer can have multiple orders
        public List<Order> Orders { get; set; }
    }
}
```

Order Entity:

So, create a class file named **Order.cs** within the Entities folder and then copy and paste the following code.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Order
    {
        public int OrderId { get; set; } // Primary Key
        public string ProductName { get; set; } // Name of the product
        public int Quantity { get; set; } // Quantity ordered
        public bool IsDeleted { get; set; } // Soft delete flag
        public DateTime OrderDate { get; set; } // Date of the order
        public int CustomerId { get; set; } // Foreign key for the customer
        public Customer Customer { get; set; } //Navigation Property
    }
}
```

Configure the Global Query Filter in DbContext

We will define a global query filter that restricts data access based on the current Customer ID. We will inject the current customer ID into the context object for demonstration purposes to filter data accordingly. So, modify the **EFCoreDbContext** class as follows:

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        // Simulated Customer ID (In real applications, retrieve this from the
        context)
        private readonly int _currentCustomerId;
        public EFCoreDbContext(int currentCustomerId)
        {
            _currentCustomerId = currentCustomerId;
        }
    }
}
```

```

    }
    public EFCoreDbContext()
    {
    }
    // Configuring the database connection and logging
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        // Display the generated SQL queries in the console
        // optionsBuilder.LogTo(Console.WriteLine, LogLevel.Information);
        // Configuring the database connection
        optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB;Trusted_Connection=True;TrustServerCertificate=True");
    }
    // Defining the global query filter in OnModelCreating
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Global Query Filter to enforce customer-based data isolation
        modelBuilder.Entity<Order>().HasQueryFilter(c => c.CustomerId == _
    }
    // DbSet representing Customers and Orders tables
    public DbSet<Order> Orders { get; set; }
    public DbSet<Customer> Customers { get; set; }
}
}

```

Apply Migrations and Update the Database

Before proceeding further, please delete the existing EFCoreDB database and the Migration folder from the Projects. Then, open the Package Manager Console and execute the following code:

- **Add-Migration AddTenantIdToCustomer**
- **Update-Database**

Insert Sample Data

Execute the following SQL commands to insert sample data into the Customers and Orders table. Ensure that orders belong to different customers.

```

USE EFCoreDB
GO
-- Insert into Customers table
INSERT INTO Customers (Name, Email) VALUES ('Alice Johnson',
'alice@example.com');
INSERT INTO Customers (Name, Email) VALUES ('Bob Smith', 'bob@example.com');
INSERT INTO Customers (Name, Email) VALUES ('Charlie Brown',
'charlie@example.com');
GO
-- Insert into Orders table
INSERT INTO Orders (ProductName, Quantity, IsDeleted, OrderDate, CustomerId)
VALUES

```

```

('Laptop', 2, 0, GETDATE(), 1),
('Smartphone', 5, 1, GETDATE(), 2),
('Headphones', 10, 0, GETDATE(), 1),
('Monitor', 3, 1, GETDATE(), 2),
('Desktop', 2, 0, GETDATE(), 1),
('Tablet', 5, 1, GETDATE(), 3),
('Keyboard', 10, 1, GETDATE(), 1),
('Earpad', 3, 1, GETDATE(), 2);
GO

```

Querying Entities with Global Query Filter Applied

When querying Orders, the global query filter automatically restricts the results to those belonging to the current customer (CustomerId = 1).

```

using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        public static void Main(string[] args)
        {
            try
            {
                // Simulating CustomerId from authentication
                int customerId = 1; // Example CustomerId
                using var context = new EFCoreDbContext(customerId);
                // Retrieve Orders for the current tenant
                var customer = context.Customers
                    .Include(ord => ord.Orders)
                    .FirstOrDefault();
                if (customer != null)
                {
                    Console.WriteLine($"Customer ID: {customerId}, Name: {customer.Name}, Email: {customer.Email}");
                    Console.WriteLine($"Customer Orders");
                    foreach (var order in customer.Orders)
                    {
                        Console.WriteLine($"  \tOrder ID: {order.OrderId}, Product Name: {order.ProductName}, Order Date: {order.OrderDate.ToShortDateString()}");
                    }
                }
                else
                {
                    Console.WriteLine($"Customer ID: {customerId} Not Found");
                }
            }
        }
    }
}

```



```

        catch (DbUpdateException ex)
        {
            Console.WriteLine($"Database error:
{ex.InnerException?.Message ?? ex.Message}");
        }
        catch (Exception ex)
        {
            Console.WriteLine($"An error occurred while fetching
cutsomer-specific orders: {ex.Message}");
        }
    }
}
}

```

Output:

```

Customer ID: 1, Name: Alice Johnson, Email: alice@example.com
Customer Orders
    Order ID: 1, Product Name: Laptop, Order Date: 26-09-2024
    Order ID: 3, Product Name: Headphones, Order Date: 26-09-2024
    Order ID: 5, Product Name: Desktop, Order Date: 26-09-2024
    Order ID: 7, Product Name: Keyboard, Order Date: 26-09-2024

```

Combining Multiple Global Query Filters in EF Core:

We can combine multiple global query filters using logical operators in the `HasQueryFilter()` method to meet complex business logic. For a better understanding, modify the `EFCoreDbContext` class as follows. Here, we are applying SOFT DELETE and CUSTOMER ID using logical AND Operators:

```

using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        // Simulated Customer ID (In real applications, retrieve this from the
        context)
        private readonly int _currentCustomerId;
        public EFCoreDbContext(int currentCustomerId)
        {
            _currentCustomerId = currentCustomerId;
        }
        public EFCoreDbContext()
        {
        }
        // Configuring the database connection and logging
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Display the generated SQL queries in the console
            // optionsBuilder.LogTo(Console.WriteLine, LogLevel.Information);
        }
    }
}

```

```

        // Configuring the database connection
        optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB;Trusted_Connection=True;TrustServerCertificate=True;")
    }
    // Defining the global query filter in OnModelCreating
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Global Query Filter to enforce customer-based data isolation

        //Combining using AND (&&) or OR (||)
        modelBuilder.Entity<Order>().HasQueryFilter(o => !o.IsDeleted && o.CustomerId == _currentCustomerId);
    }
    // DbSet representing Customers and Orders tables
    public DbSet<Order> Orders { get; set; }
    public DbSet<Customer> Customers { get; set; }
}
}

```

Output:

```

Customer ID: 1, Name: Alice Johnson, Email: alice@example.com
Customer Orders
    Order ID: 1, Product Name: Laptop, Order Date: 26-09-2024
    Order ID: 3, Product Name: Headphones, Order Date: 26-09-2024
    Order ID: 5, Product Name: Desktop, Order Date: 26-09-2024

```

Note: Global Query Filters are not applied to raw SQL queries executed via methods like `FromSqlRaw`.

Limitations and Considerations Global Query Filters in Entity Framework Core

- **Filters Don't Apply to `FromSqlRaw/FromSqlInterpolated`:** Global query filters do not apply automatically when we use raw SQL queries via `FromSqlRaw` or `FromSqlInterpolated`. If you rely heavily on raw SQL, you may have to replicate some filtering logic directly in your SQL statements.
- **Query Performance:** Each global filter adds a WHERE clause to your queries for the specified entity. If you have many filters or complex filters, it might impact performance. Proper indexing (e.g., indexing on the `IsDeleted` and `TenantId` columns) can help maintain good performance.
- **Navigation Properties and Include:** Global filters also apply to navigation properties when you use `.Include(...)`. If you load an entity with includes, the child entities will be filtered unless we disable or override the filter.

Global Query Filters in Entity Framework Core help us implement consistent filtering logic across all queries without the need for repetitive code. They are particularly useful for scenarios like soft deletes, multi-tenancy, and access control. However, it's essential to keep performance and maintainability in mind. They can help keep our application code cleaner, more maintainable, and consistent when used effectively.

In the next article, I will discuss the [Entity Framework Core Database First Approach](#) with Examples. In this article, I try to explain **Global Query Filters in Entity Framework Core** (EF Core) with Examples. I hope you enjoy this article on Global Query Filters in Entity Framework Core.